

Ridge Regression: The Maths and Code from Scratch

Contents

1. Overview	2
2. One input	2
2.1 The loss	2
2.2 The intercept	3
2.3 The slope	3
2.4 Why the slope shrinks	3
3. Many inputs	5
3.1 The loss in matrix form	5
3.2 Expanding and differentiating	5
3.3 Solving for w	5
3.4 Not penalising the intercept	6
4. Summary	8
5. Sources	8
6. Key terms	8

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Standardization ([Note 24](#)); Multiple linear regression ([Note 55](#)); Normal equation ([Note 55](#)); Gradient descent ([Note 57](#)); Bias-variance trade-off ([Note 62](#)); Regularisation ([Note 63](#)).
- **Leads to:** Elastic Net ([Note 69](#)).
- **Compare with:** Lasso regression ([Note 67](#)); L1 and L2 regularisation in neural networks ([Note 1026](#)).

1. Overview

Key point: Adding the Ridge penalty to the loss and setting the derivative to zero gives a closed-form answer. With one input the slope becomes $m = \frac{\sum (x-\bar{x})(y-\bar{y})}{\sum (x-\bar{x})^2 + \lambda}$; with many inputs it becomes $\beta = (X^T X + \lambda I)^{-1} X^T y$.

The previous Note explained what Ridge does: it adds λ times the squared coefficients to the loss, which keeps them small. This Note derives the formulas that find the Ridge coefficients, the same way the OLS and normal equation Notes did for plain linear regression.

There are two cases:

- **One input:** a formula for the slope m and the intercept b .
- **Many inputs:** a matrix formula for all coefficients at once.

Both are then coded from scratch and checked against scikit-learn's **Ridge**.

2. One input

Key point: The Ridge slope is the OLS slope with λ added to the bottom of the fraction. The intercept formula does not change.

2.1 The loss

Key point: The loss is the usual sum of squared errors plus λm^2 .

With one input, the prediction is $\hat{y}_i = mx_i + b$. The Ridge loss is

$$L = \sum_{i=1}^n (y_i - mx_i - b)^2 + \lambda m^2$$

Only the slope m is penalised, not the intercept b .

2.2 The intercept

Key point: The penalty does not contain b , so the derivative with respect to b is the same as in OLS, and so is the answer.

Differentiate L with respect to b and set it to zero. The term λm^2 does not contain b , so it disappears:

$$\frac{\partial L}{\partial b} = -2 \sum_{i=1}^n (y_i - mx_i - b) = 0$$

This is exactly the OLS step, and it gives the same result:

$$b = \bar{y} - m\bar{x}$$

2.3 The slope

Key point: Differentiating with respect to m adds a $2\lambda m$ term, which ends up as $+\lambda$ in the denominator.

Differentiate L with respect to m . The squared-error part gives the OLS terms; the penalty adds $2\lambda m$:

$$\frac{\partial L}{\partial m} = -2 \sum_{i=1}^n (y_i - mx_i - b) x_i + 2\lambda m = 0$$

Substitute $b = \bar{y} - m\bar{x}$ and rearrange, as in the OLS Note. The only difference is the extra λm , which joins the other m terms:

$$m = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2 + \lambda}$$

In words: the top of the fraction is the same as in OLS, and the bottom has λ added. With $\lambda = 0$ it is the OLS slope.

Extra: Strictly, substituting b turns $\sum (y_i - \bar{y})x_i$ into $\sum (y_i - \bar{y})(x_i - \bar{x})$ and $\sum (x_i - \bar{x})x_i$ into $\sum (x_i - \bar{x})^2$. These are equal because $\sum (y_i - \bar{y}) = 0$ and $\sum (x_i - \bar{x}) = 0$, the same trick as in the OLS derivation.

2.4 Why the slope shrinks

Key point: A bigger λ makes the denominator bigger, so the slope gets smaller. It approaches 0 but never reaches it.

On the 100-point example from the previous Note, the top of the fraction is 2416.7 and the bottom without λ is 86.85. So

$$m = \frac{2416.7}{86.85 + \lambda}$$

λ	Slope m	Intercept b
0	$2416.7/86.85 = 27.83$	-2.30
10	$2416.7/96.85 = 24.95$	-2.13
100	$2416.7/186.85 = 12.93$	-1.43

These are exactly the slopes scikit-learn gave in the previous Note. Figure 1 shows the whole curve.

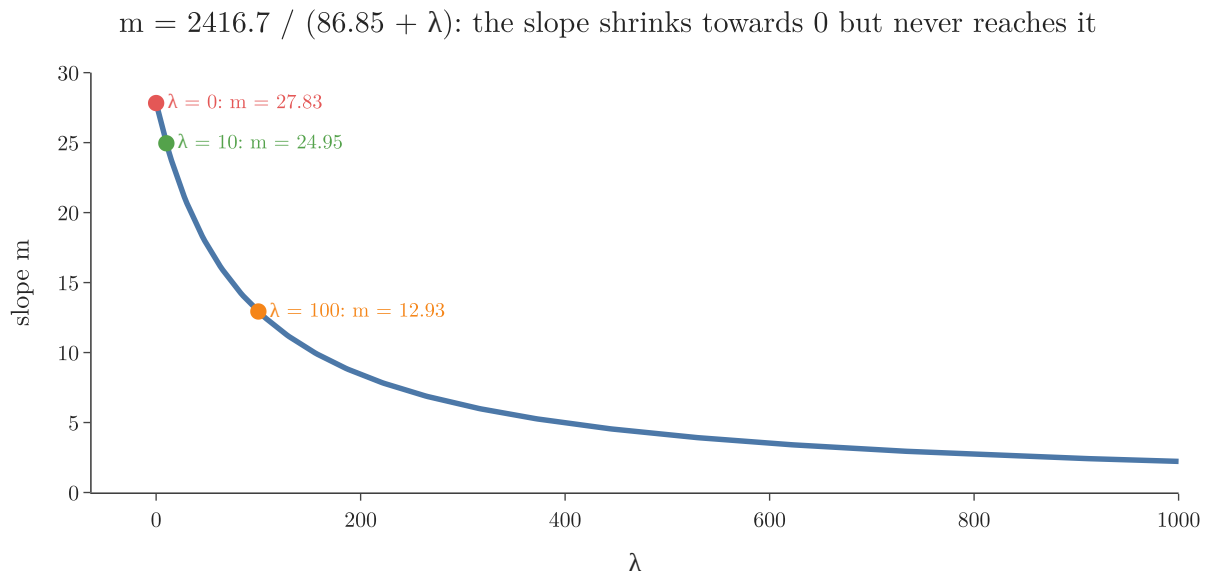


Figure 1: The Ridge slope against λ

However large λ gets, the fraction stays above 0. So Ridge makes coefficients small but never exactly zero. Lasso, two Notes later, behaves differently.

Python: Ridge with one input, from scratch.

```
class MyRidge:
    def __init__(self, alpha=0.1):
        self.alpha = alpha  # the  $\lambda$  of the formula

    def fit(self, x, y):
        num = np.sum((x - x.mean()) * (y - y.mean()))
        den = np.sum((x - x.mean()) ** 2) + self.alpha
        self.m = num / den
        self.b = y.mean() - self.m * x.mean()
```

```
return self
```

```
MyRidge(alpha=10).fit(x, y).m # 24.955, same as Ridge(alpha=10)
```

3. Many inputs

Key point: In matrix form the Ridge answer is the normal equation with λI added inside the inverse.

3.1 The loss in matrix form

Key point: The penalty λ times the sum of squared coefficients is written $\lambda w^T w$.

With many inputs, the predictions are $\hat{y} = Xw$, where X has a first column of 1s and w holds the intercept and all coefficients (the normal equation Note). The sum of squared coefficients is $w^T w$, so the Ridge loss is

$$L = (y - Xw)^T (y - Xw) + \lambda w^T w$$

3.2 Expanding and differentiating

Key point: The squared-error part gives the same derivative as before; the penalty adds $2\lambda w$.

Expanding the first part exactly as in the normal equation Note gives

$$L = y^T y - 2w^T X^T y + w^T X^T X w + \lambda w^T w$$

Differentiate with respect to w , using the rules from that Note ($w^T w$ behaves like w^2 , with derivative $2w$):

$$\frac{\partial L}{\partial w} = -2X^T y + 2X^T X w + 2\lambda w = 0$$

3.3 Solving for w

Key point: Collect the w terms, factor out w , and multiply by the inverse.

Divide by 2 and move $X^T y$ to the right:

$$X^T X w + \lambda w = X^T y$$

To factor out w , write λw as $\lambda I w$, where I is the identity matrix (a matrix cannot be added to a number):

$$(X^T X + \lambda I) w = X^T y$$

Multiply both sides by the inverse of the bracket:

$$w = (X^T X + \lambda I)^{-1} X^T y$$

$$\beta = (X^T X + \lambda I)^{-1} X^T y \quad I = \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}$$

compared with the normal equation, only
 $+\lambda I$ is new; it adds λ to the diagonal

top-left entry 0: the
intercept β_0 is not penalised

Figure 2: The Ridge formula and its identity matrix

Figure 2 shows the result. Compared with the normal equation, only $+\lambda I$ is new: it adds λ to every number on the diagonal of $X^T X$.

3.4 Not penalising the intercept

Key point: The top-left entry of I is set to 0, so the intercept is left out of the penalty.

The first entry of w is the intercept. It only shifts predictions up or down, so it should not be penalised (the previous Note). Setting the top-left entry of I to 0 removes it from the penalty, as in Figure 2.

Python: Ridge with many inputs, from scratch.

```
class MyRidgeND:
    def __init__(self, alpha=0.1):
        self.alpha = alpha

    def fit(self, X, y):
        X = np.insert(X, 0, 1, axis=1)           # column of 1s
        I = np.identity(X.shape[1])
        I[0, 0] = 0                               # intercept not penalised
        w = np.linalg.inv(X.T @ X + self.alpha * I) @ X.T @ y
        self.intercept_, self.coef_ = w[0], w[1:]
        return self
```

```
def predict(self, X):
    return X @ self.coef_ + self.intercept_
```

On the diabetes data (test size 0.2, random state 4) with alpha 0.1:

	Intercept	Test R^2
MyRidgeND	150.89	0.4693
Ridge(alpha=0.1, solver="cholesky")	150.89	0.4693

The coefficients agree to within 3×10^{-12} , rounding error.

Extra: scikit-learn reaches the same answer differently: it centres X and y (subtracts their means), solves for the coefficients without an intercept column, and then computes the intercept from the means. That is the matrix version of $b = \bar{y} - m\bar{x}$. The "cholesky" solver then solves the same equation $(X^T X + \lambda I)w = X^T y$ directly, which is why the two rows of the table above agree (scikit-learn docs, `Ridge`).

Extra: A bonus of $+\lambda I$: when columns are strongly correlated (multicollinearity), $X^T X$ can be impossible or unstable to invert. Adding a positive λ to the diagonal always makes it invertible, so Ridge has an answer even when plain OLS does not. In fact this was the main reason Ridge was first introduced (ESL §3.4.1). The maths in one line: for any non-zero vector v , $v^T (X^T X + \lambda I) v = \|Xv\|^2 + \lambda \|v\|^2 > 0$, so no v is sent to zero and the matrix has an inverse.

4. Summary

	Linear regression (OLS)	Ridge
Loss	$\sum (y_i - \hat{y}_i)^2$	$\sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2$
Slope, one input	$\frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2}$	$\frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2 + \lambda}$
Intercept, one input	$\bar{y} - m\bar{x}$	$\bar{y} - m\bar{x}$
Many inputs	$(X^T X)^{-1} X^T y$	$(X^T X + \lambda I)^{-1} X^T y$

- Ridge adds λ to the denominator of the slope (one input) or to the diagonal of $X^T X$ (many inputs).
- A larger λ gives smaller coefficients, but never exactly 0.
- The intercept is not penalised: the top-left entry of I is 0.
- The from-scratch code matches scikit-learn's `Ridge` exactly.

5. Sources

- **ESL:** Hastie, T., Tibshirani, R. and Friedman, J. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009. Section 3.4.1, p. 64 (which credits Hoerl and Kennard, 1970).
- **scikit-learn docs:** `sklearn.linear_model.Ridge` (`fit_intercept`, `solver="cholesky"`), scikit-learn 1.9 documentation and source (`_preprocess_data`).

6. Key terms

Term	Meaning
Identity matrix	A square matrix with 1s on the diagonal and 0s elsewhere; multiplying by it changes nothing
Closed-form solution	A formula that gives the answer directly, without repeated steps
Cholesky solver	A scikit-learn Ridge solver that solves the closed-form equation directly